

03 · Arquitectura

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 03-architecture.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Estilo arquitectónico
2. Capas del sistema
3. Diagrama de componentes
4. Mapa mental del sistema
5. Flujo principal: una captura
6. Patrones de diseño identificados
7. Procesos síncronos y asíncronos
8. Manejo de estado
9. Manejo de errores
10. Autenticación y autorización
11. Persistencia y caché
12. Procesos en segundo plano
13. Diagrama de despliegue
14. Diagrama entidad-relación
15. Ediciones de compilación
16. Decisiones arquitectónicas y sus consecuencias

Cómo está construido el sistema y por qué así. Cada diagrama va acompañado de su explicación en texto: el diagrama ayuda, pero la información está en la prosa.

1. Estilo arquitectónico

RootCause macOS Inspector es una **aplicación monolítica de escritorio con arquitectura en capas y un adaptador de sistema operativo aislado**. No hay servidor, ni procesos auxiliares, ni comunicación entre máquinas: un solo binario que puede presentarse como interfaz gráfica o como comando.

Cinco decisiones definen el estilo:

1. **Una única fuente de verdad por captura.** Toda la información de un instante vive en un `SystemSnapshot`. La interfaz, el CLI, el exporte JSON, el reporte Markdown y el historial consumen ese mismo objeto; ninguno vuelve a consultar el sistema por su cuenta.
2. **Todo el contacto con macOS pasa por un módulo.** `services/macros.rs` es el único que ejecuta procesos externos. El resto del código recibe texto y lo parsea. Eso hace que los parsers sean funciones puras y, por tanto, comprobables sin un Mac detrás.
3. **Recolectar y decidir son trabajos distintos.** `services/inspector.rs` recoge; `services/rules.rs` y `services/anomaly.rs` deciden qué significa. Mezclarlos haría imposible probar el segundo.
4. **La interfaz nunca bloquea.** El motor vive en un hilo propio y se comunica por canales `std::sync::mpsc`. Una captura lenta no congela la ventana.
5. **Fallo suave por superficie.** Si `lsof` no está, si `TCC.db` no se puede leer o si `spctl` no responde, esa sección queda vacía y se explica; la captura sigue siendo útil.

2. Capas del sistema

Diagrama Mermaid #1 — código fuente (no renderizado en PDF)

```
graph TD
    subgraph P ["Presentación"]
        GUI["app.rs · interfaz equi<br/>12 secciones"]
        CLI["cli.rs · 19 comandos"]
    end
    subgraph O ["Orquestación"]
        INS["services/inspector.rs<br/>InspectorService"]
    end
    subgraph D ["Dominio y decisión"]
        RUL["services/rules.rs"]
        ANO["services/anomaly.rs"]
        BAS["services/baseline.rs"]
        MOD["models.rs · 33 tipos"]
    end
    subgraph R ["Recolección por superficie"]
        LAU["launchd.rs"]
        SEC["security.rs"]
        TCC["tcc.rs"]
        NET["network.rs"]
        NSC["netscan.rs"]
        TMP["temp_scan.rs"]
        RES["resilience.rs"]
    end
    subgraph A ["Adaptador de sistema"]
        MAC["services/macros.rs"]
    end
    subgraph I ["Infraestructura"]
        DB["persistence.rs · SQLite"]
        CFG["config.rs · JSON"]
        REP["report.rs · Markdown"]
        AI["ai.rs · IA opcional"]
    end
    subgraph S ["macOS"]
        OS["lsof · spctl · csrutil · fdesetup<br/>codesign · arp · launchctl · ps · log"]
        FS["Archivos: plists · TCC.db · cachés"]
    end
    GUI --> INS
```

```

CLI --> INS
INS --> RUL
INS --> ANO
INS --> BAS
INS --> LAU
INS --> SEC
INS --> TCC
INS --> NET
INS --> NSC
INS --> TMP
INS --> RES
INS --> DB
INS --> CFG
INS --> REP
INS --> AI
LAU --> MAC
SEC --> MAC
NET --> MAC
NSC --> MAC
BAS --> DB
MAC --> OS
TCC --> FS
TMP --> FS
LAU --> FS
RUL --> MOD
ANO --> MOD

```

2.1 Responsabilidad de cada capa

Capa	Archivos	Responsabilidad	Qué no hace
Presentación	<code>app.rs, cli.rs</code>	Dibujar, formatear, recoger la intención del usuario	No ejecuta comandos del sistema ni decide severidades
Orquestación	<code>services/inspector.rs</code>	Secuenciar la captura, aplicar baselines, persistir, exponer acciones	No parsea salidas ni pinta
Dominio y decisión	<code>rules.rs, anomaly.rs, baseline.rs, models.rs</code>	Clasificar, puntuar, correlacionar, comparar contra baseline	No toca el sistema operativo ni el disco
Recolección	<code>launchd.rs, security.rs, tcc.rs, network.rs, netscan.rs, temp_scan.rs, resilience.rs</code>	Convertir una superficie del sistema en modelos de dominio	No decide el veredicto global
Adaptador	<code>services/macros.rs</code>	Ejecutar utilidades nativas y devolver texto	No interpreta el significado de ese texto
Infraestructura	<code>persistence.rs, config.rs, report.rs, ai.rs</code>	SQLite, JSON de configuración, Markdown, petición HTTPS opcional	No participa en la clasificación

2.2 Regla de dependencias

Las dependencias apuntan hacia abajo y hacia el centro: la presentación depende de la orquestación, la orquestación de todo lo demás, y el dominio no depende de nadie salvo de los modelos. **`models.rs` no importa ningún otro módulo del proyecto**, lo que lo convierte en el vocabulario común de todas las capas.

Excepciones reales, verificadas en el código:

- `services/security.rs` llama a `crate::services::launchd::loaded_labels()` para deducir si SSH está activo. Es una dependencia lateral entre recolectores, documentada en el propio código porque `systemsetup -getremotelogin` exige root.
- `services/anomaly.rs` usa `crate::services::network` para extraer IP y agrupar destinos. Reutiliza el parser en vez de duplicarlo.

- `services/inspector.rs` usa `crate::services::baseline`, que a su vez usa `PersistenceStore`: el motor de baseline necesita leer la foto anterior.

3. Diagrama de componentes

Diagrama Mermaid #2 — código fuente (no renderizado en PDF)

```
flowchart LR
    subgraph Entrada
        MAIN["main.rs"]
    end
    MAIN -->|"argumentos"| CLI["cli.rs"]
    MAIN -->|"sin argumentos<br/>y feature gui"| APP["app.rs"]
    APP -->|"Command"| WORKER["hilo de trabajo"]
    WORKER -->|"EngineEvent"| APP
    WORKER --> SVC["InspectorService"]
    CLI --> SVC
    SVC --> COL["Recolectores"]
    SVC --> DEC["Reglas y anomalías"]
    SVC --> STORE["PersistenceStore"]
    SVC --> CFG["ConfigManager"]
    SVC --> RESIL["ResilienceMonitor"]
    SVC -->|"solo bajo demanda"| AI["AiAdvisor"]
    STORE --> SQLITE[("rootcause-history.db")]
    CFG --> JSON[("rootcause-config.json")]
    RESIL --> STATE[("rootcause-agent-state.json")]
```

Lectura: `main.rs` es un despachador de tres líneas. Con argumentos (y salvo `--gui`) entrega el control al CLI y sale con su código de retorno; sin argumentos abre la ventana. La interfaz **no posee** el motor: lo posee un hilo de trabajo con el que habla por dos canales tipados, `Command` hacia el motor y `EngineEvent` de vuelta. El CLI, en cambio, posee el motor directamente porque su ejecución es de un solo disparo.

4. Mapa mental del sistema

Diagrama Mermaid #3 — código fuente (no renderizado en PDF)

```
mindmap
    root((("RootCause<br/>macOS Inspector")))
        ("Qué observa")
            ("Persistencia<br/>launchd · cron · login items")
            ("Procesos<br/>consumo · firma · ruta")
            ("Controles nativos<br/>Gatekeeper · SIP · FileVault")
            ("XProtect<br/>versión y antigüedad")
            ("Privacidad<br/>TCC.db")
            ("Red<br/>conexiones · vecinos ARP")
            ("Almacenamiento<br/>cachés y temporales")
        ("Cómo decide")
            ("Puntaje acumulativo<br/>ninguna señal decide sola")
            ("Sostenido, no instantáneo<br/>rachas por PID")
            ("Baseline<br/>NUEVA · MODIFICADA · ELIMINADA")
            ("Correlación<br/>incidente con evidencia")
        ("Cómo se presenta")
            ("GUI · 12 secciones")
            ("CLI · 19 comandos")
            ("JSON · captura completa")
            ("Markdown · reporte forense")
        ("Qué nunca hace")
            ("Telemetría")
            ("Acciones automáticas")
            ("Eliminar malware")
            ("Escalar privilegios")
```

5. Flujo principal: una captura

Diagrama Mermaid #4 — código fuente (no renderizado en PDF)

```
sequenceDiagram
    participant U as Usuario
    participant A as app.rs (GUI)
    participant W as Hilo de trabajo
    participant I as InspectorService
```

```

participant M as macos.rs
participant B as baseline.rs
participant R as rules.rs
participant S as PersistenceStore

U->>A: abre la app / pulsa Actualizar / F5
A->>W: Command::Refresh
W->>I: collect_snapshot()
I->>I: resilience_monitor.heartbeat()
I->>M: process_details() · lsof_connections()
M->>I: texto crudo
I->>I: collect_processes() + apply_signatures()
I->>M: code_signature(ruta) para el presupuesto
I->>I: temp_scan · security · tcc · launchd · netscan
I->>B: diff_surface(superficie, items)
B->>S: load_baseline(surface)
S->>B: baseline previa
B->>I: items marcados Added/Modified/Removed
I->>I: anomaly_tracker.analyze()
I->>R: build_alerts(...)
R->>I: alertas ordenadas + veredicto
I->>R: derive_incident(snapshot)
R->>I: incidente o None
I->>S: persist_incident() · persist_snapshot()
I->>W: SystemSnapshot
W->>A: EngineEvent::Snapshot
A->>U: repinta con el nuevo estado

```

Puntos que el diagrama no muestra y conviene saber:

- El latido de resiliencia se ejecuta **antes** de nada, para que un cierre abrupto de la sesión anterior se refleje en esta captura.
- La verificación de firma no se aplica a todos los procesos: hay un presupuesto (`signature_budget`, 12 por defecto) y una caché por ruta, porque `codesign` cuesta un proceso por llamada.
- Tras resolver firmas, los procesos afectados **se reclasifican**: la firma es una señal fuerte y debe entrar en el puntaje.
- Si la persistencia en SQLite falla, no se pierde la captura: se añade una alerta de advertencia y la vista sigue funcionando.

6. Patrones de diseño identificados

Patrón	Dónde	Por qué
Adapter	<code>services/macos.rs</code>	Aísla la dependencia del sistema operativo en un punto
Facade	<code>InspectorService</code>	Una sola superficie para GUI y CLI sobre siete recolectores
Strategy ligera	<code>SurfaceSpec</code> en <code>baseline.rs</code>	El mismo motor de diff sirve a cuatro superficies cambiando una constante
Command / Event	<code>Command</code> y <code>EngineEvent</code> en <code>app.rs</code>	Desacopla la interfaz del motor y evita bloqueos
Repository	<code>PersistenceStore</code>	Encapsula SQLite; el resto del código no ve SQL
Value Object	<code>WatchedItem</code> , <code>IncidentEvidence</code>	Unidades comparables y serializables sin identidad propia
Null Object explícito	<code>TccOverview::readable = false</code>	La ausencia de dato es un dato, no una lista vacía
Guard clause	Todos los parsers	<code>let ... else { continue }</code> en vez de anidar

7. Procesos síncronos y asíncronos

El proyecto **no usa `async/await` ni ningún runtime asíncrono**. La concurrencia se resuelve con un hilo y canales:

Ejecución	Modelo	Detalle
CLI	Síncrono, un disparo	<code>cli::run</code> crea el servicio, captura y sale con un código
GUI	Un hilo de interfaz + un hilo de motor	<code>spawn_worker</code> en <code>app.rs</code>
Barrido de red profundo	Síncrono dentro del hilo de motor	254 <code>ping</code> en serie con <code>-t 1</code>
Log unificado	Síncrono bajo demanda	<code>log show</code> tarda segundos; nunca entra en la captura periódica
Petición IA	Síncrona, proceso hijo <code>curl</code>	Con <code>--max-time</code> configurable

La GUI pide repintado con `ctx.request_repaint_after(120 ms)` mientras hay trabajo y `900 ms` en reposo, y el motor llama a `ctx.request_repaint()` cada vez que responde.

8. Manejo de estado

Estado	Dónde vive	Duración
Última captura	<code>RootCauseApp::snapshot</code>	Hasta la siguiente
Series de tendencia (CPU, memoria, escritura)	<code>Vec<f32></code> con tope de 60 muestras	Sesión
Deltas de E/S por PID	<code>InspectorService::process_baselines</code>	Sesión; se purgan los PID muertos
Caché de firmas por ruta	<code>InspectorService::signature_cache</code>	Sesión
Rachas por PID y reapariciones	<code>AnomalyTracker</code>	Sesión
Baselines de superficie	Tablas <code>baseline</code> y <code>persistence_baseline</code>	Permanente hasta aceptar otra
Historial e incidentes	Tablas <code>snapshots</code> e <code>incidents</code>	Recortado a <code>history_limit</code> / <code>incident_limit</code>
Salud del agente	<code>rootcause-agent-state.json</code>	Entre ejecuciones
Idioma activo	<code>AtomicU8</code> global en <code>i18n.rs</code>	Proceso
Paleta activa	<code>static mut ACTIVE_DARK</code> en <code>app.rs</code>	Proceso, solo hilo de interfaz

9. Manejo de errores

Tres niveles, deliberadamente distintos:

1. **Errores recuperables por superficie.** Un `Result` que se descarta con `unwrap_or_default()` y deja la sección vacía. Ejemplo: `macos::arp_table()` falla y `netscan::scan` devuelve cero dispositivos con sus limitaciones declaradas.

2. **Errores que el usuario debe ver.** Se propagan con `anyhow::Result` y contexto (`.context("No se pudo guardar el reporte")`), y acaban en la barra de estado de la GUI o en `stderr` del CLI con código de salida 1.
3. **Errores que no deben ocurrir.** No hay `unwrap()` sobre entradas externas en la ruta de captura; los `expect` viven en los tests.

Además, los errores de una acción manual **se auditan igual que los éxitos**: `InspectorService::audit` recibe `Option<anyhow::Error>` y guarda el detalle.

10. Autenticación y autorización

No existen dentro del producto: **no hay cuentas, sesiones, roles ni tokens propios**. La autorización es la del sistema operativo, y el código la trata como un hecho observable:

- `macos::environment()` declara usuario, UID, si es root y qué utilidades faltan.
- `tcc::scan()` declara si pudo leer las bases y si hay Acceso total al disco.
- `InspectorService::can_terminate_process` aplica una **política local** por encima de la del sistema: nunca el propio proceso, nunca $PID \leq 1$, nunca uno de los trece nombres protegidos (`kernel_task`, `launchd`, `windowserver`, `loginwindow`, `opendirectoryd`, `securityd`, `syspolicyd`, `configd`, `powerd`, `hidd`, `coreaudiod`, `notifyd`, `diskarbitrationd`).

11. Persistencia y caché

Diagrama Mermaid #5 — código fuente (no renderizado en PDF)

```
flowchart LR
    SNAP["SystemSnapshot"] -->|"resumen compacto"| T1["snapshots"]
    SNAP -->|"si hay incidente"| T2["incidents"]
    ACC["Acciones manuales"] --> T3["audit_log"]
    PERS["Entradas de persistencia"] --> T4["persistence_baseline"]
    WATCH["WatchedItem de 3 superficies"] --> T5["baseline"]
    T1 --- DB1["rootcause-history.db<br/>SQLite"]
    T2 --- DB2["DB"]
    T3 --- DB3["DB"]
    T4 --- DB4["DB"]
    T5 --- DB5["DB"]
```

Cinco tablas en un único archivo SQLite. No hay ORM: `rusqlite` con SQL literal en `persistence.rs`. Las tablas de volumen (`snapshots`, `incidents`) se recortan tras cada inserción con `trim_table`, así que el archivo no crece sin límite. Esquema completo en [07 · Base de datos](#).

Cachés en memoria: firmas por ruta, deltas de E/S por PID e historial de rachas. Ninguna sobrevive al proceso, por decisión: una caché de firmas persistida podría dar por buena la firma de un binario que fue sustituido.

12. Procesos en segundo plano

Proceso	Disparo	Coste
Hilo de motor (GUI)	Al arrancar la app	Vive hasta <code>Command::Shutdown</code>
Refresco automático	Temporizador de <code>refresh_interval_secs</code> (mínimo forzado 2 s)	Una captura
Latido de resiliencia	Dentro de cada captura, con intervalo propio (15 s)	Escritura de un JSON pequeño
Notificación crítica	Alerta crítica + <code>notify_on_critical</code>	Un <code>osascript</code>
Barrido profundo de red	Solo botón «Escaneo profundo» o <code>network --deep</code>	254 <code>ping</code> en serie

Log unificado	Solo sección Historial o <code>events</code>	Segundos de <code>log show</code>
---------------	--	-----------------------------------

No hay demonio ni LaunchAgent propio: RootCause no se instala como servicio y no se ejecuta si nadie lo abre. Es una decisión de producto coherente con su posicionamiento —una herramienta que vigila la persistencia ajena no debería añadir la suya sin decirlo.

13. Diagrama de despliegue

Diagrama Mermaid #6 — código fuente (no renderizado en PDF)

```
flowchart TB
    subgraph DEV["Equipo de desarrollo"]
        SRC["Código fuente"] --> CARGO["cargo build --release"]
        CARGO --> BIN["target/release/rootcause"]
        BIN --> PKG["scripts/package-app.sh"]
        PKG --> APPB["dist/RootCause.app"]
        APPB --> DMG["scripts/package-dmg.sh<br/>dist/RootCause-0.1.0.dmg"]
    end
    subgraph CI["GitHub Actions"]
        W1["ci.yml<br/>markdownlint · fmt · clippy · test · build"]
        W2["release-macos.yml<br/>universal · .app · .dmg · SHA256SUMS"]
        W3["deploy-landing.yml<br/>GitHub Pages"]
    end
    subgraph USER["Mac del usuario"]
        INST["~/Applications/RootCause.app<br/>o binario en PATH"]
        DATA["~/Library/Application Support/<br/>RootCauseInspector/"]
        REPORTS["~/Documents/RootCause/reports/"]
        INST --> DATA
        INST --> REPORTS
    end
    DMG -->|"release manual o por tag"| USER
    W2 -->|"artefactos de la release"| USER
```

El despliegue es una descarga o una compilación local: no hay servidor, ni actualizador automático, ni canal de distribución propio. Los binarios publicados **no están firmados ni notarizados**, lo que el propio cuerpo de la release advierte.

14. Diagrama entidad-relación

El modelo de datos persistido es deliberadamente plano. Detalle y diccionario de datos en [07 · Base de datos](#).

Diagrama Mermaid #7 — código fuente (no renderizado en PDF)

```
erDiagram
    SNAPSHOTS {
        INTEGER id PK
        TEXT collected_at
        REAL cpu_usage
        TEXT dominant_process
        TEXT alerts_json
    }
    INCIDENTS {
        INTEGER id PK
        TEXT incident_id
        TEXT fingerprint
        TEXT severity
        TEXT payload_json
    }
    AUDIT_LOG {
        INTEGER id PK
        TEXT occurred_at
        TEXT action
        TEXT target
        INTEGER success
    }
    PERSISTENCE_BASELINE {
        TEXT entry_key PK
        TEXT entry_kind
        TEXT location
        TEXT name
        TEXT command
    }
    BASELINE {
        TEXT surface PK
        TEXT entry_key PK
    }
```

```
TEXT value
TEXT label
}
```

No hay claves foráneas declaradas: las tablas son independientes por diseño. La relación entre un incidente y la captura que lo originó existe **por marca de tiempo** (`collected_at`), no por referencia. Es una simplificación consciente que se documenta como deuda técnica menor en [15 · Riesgos](#).

15. Ediciones de compilación

Diagrama Mermaid #8 — código fuente (no renderizado en PDF)

```
flowchart LR
    C["Cargo.toml<br/>features"] --> D["default = gui"]
    C --> G["gui = eframe + egui"]
    D --> FULL["cargo build --release<br/>GUI + CLI"]
    G --> FULL
    C --> NOF["--no-default-features<br/>solo CLI"]
    FULL --> APP[".app · .dmg"]
    NOF --> SRV["SSH · scripts · CI"]
```

La edición CLI-only no compila `app.rs`. Para que eso no genere avisos de código muerto en funciones compartidas, `main.rs` declara `#![cfg_attr(not(feature = "gui"), allow(dead_code))]`, con un comentario que explica que no es código muerto sino código usado por la otra edición. **La CI compila las dos** en cada push, así que ninguna se rompe en silencio.

16. Decisiones arquitectónicas y sus consecuencias

Decisión	Ventaja	Coste aceptado
Ejecutar utilidades del sistema en vez de usar APIs nativas	Cero dependencias de FFI; salida auditable que se puede mostrar como evidencia	Coste de proceso por llamada; formatos que cambian entre versiones de macOS
Traducción local con <code>tr(es, en)</code> en vez de diccionario con claves	Imposible una clave huérfana; cero asignaciones	Las cadenas se repiten en el código
Estado global para idioma y paleta	No hay que cablear parámetros por cada función de dibujo	<code>static mut</code> requiere <code>unsafe</code> y solo es válido en el hilo de interfaz
SQLite embebido (<code>bundled</code>)	No depende de la librería del sistema	Primera compilación más lenta
Sin cliente HTTP; <code>curl</code> para la IA	Ninguna dependencia nueva por una función opcional	Depende de que <code>curl</code> exista (viene con macOS)
Baseline «pegajosa»	Un cambio no se auto-silencia tras un reinicio	Requiere que el usuario acepte explícitamente
Primera baseline sembrada en silencio	Estrenar la herramienta no genera cien alertas	La primera ejecución no detecta lo que ya estaba mal

Esa última fila es la más importante del documento: **RootCause detecta cambios, no estados malos preexistentes**. Si el equipo ya estaba comprometido cuando se sembró la baseline, esa persistencia quedará marcada como conocida. El producto lo dice en su documentación y es la limitación estructural del enfoque.

Siguiente lectura recomendada: [04 · Mapa completo del código](#) para bajar al detalle de archivos y funciones.